

Software Engineering

TAHIR FAROOQ
FAST-NU

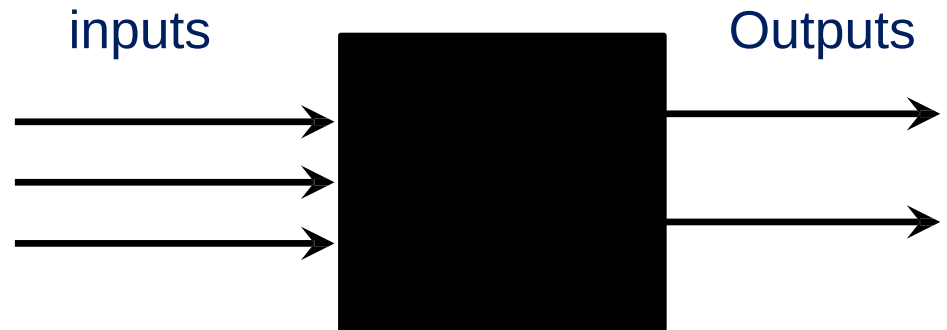
What Will You Learn Today?

- ❖ Exhaustive testing
- ❖ Equivalence class testing (Equivalence Partitioning)
- ❖ Boundary value analysis
- ❖ Decision table testing
- ❖ Pairwise testing
- ❖ State-Transition testing
- ❖ Domain analysis testing
- ❖ Use case testing

Black-Box Testing Techniques

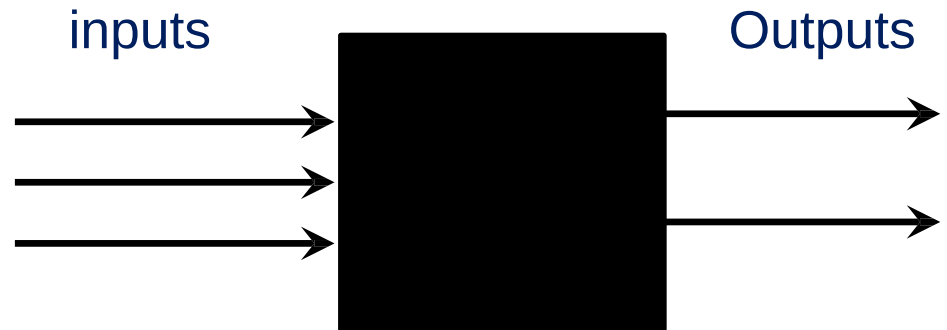
Black-Box Testing Techniques

- A strategy in which testing is based on **requirements** and **specifications**
- **Applicability:** All levels of system development
 - Unit
 - Integration
 - System
 - Acceptance



Black-Box Testing Techniques

- Incorrect or missing functions
- Interface errors
- Performance error



Black-Box Testing Techniques

- **Advantages:** directs tester to choose **subsets** to tests that are both **efficient** and **effective** in **finding defects**
- **Disadvantages:** never be sure of **how much** of the system under test (SUT) has been tested

Black-Box Testing Techniques

- ❖ Exhaustive testing
- ❖ Equivalence class testing (Equivalence Partitioning)
- ❖ Boundary value analysis
- ❖ Decision table testing
- ❖ Pairwise testing
- ❖ State-Transition testing
- ❖ Domain analysis testing
- ❖ Use case testing

Exhaustive Testing

1.Definition: testing with every member of the input value space

2.Input value space: the set of all possible input values to the program

Consider an application in which a password field that accepts **3 characters**, with no consecutive repeating entries. Hence, there are **$26 * 26 * 26$** input permutations for alphabets only. Including special characters and standard characters, there are much more combinations. So, there are **$256 * 256 * 256$** input combinations.

Equivalence Class Testing

EC Testing is when you have a **number of test items** (e.g. values) that you want to test but **because of cost** (time/money) you do not have time to test them all. Therefore you **group the test item into class** where all items in each class are suppose to behave exactly the same. **The theory is that you only need to test one of each item to make sure the system works.**

Example 1

Children under 2 ride the buss for free. Young people pay \$10, Adults \$15 and Senior Citizen pay \$5.

Classes:

Price:0 -> Age:0-1

Price:10 -> Age:2-14

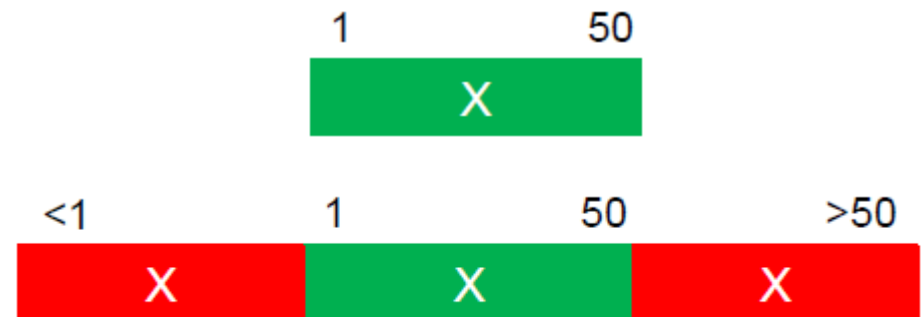
Price:15 -> Age:15-64

Price:5 -> Age:65-infinity

Equivalence Class Techniques

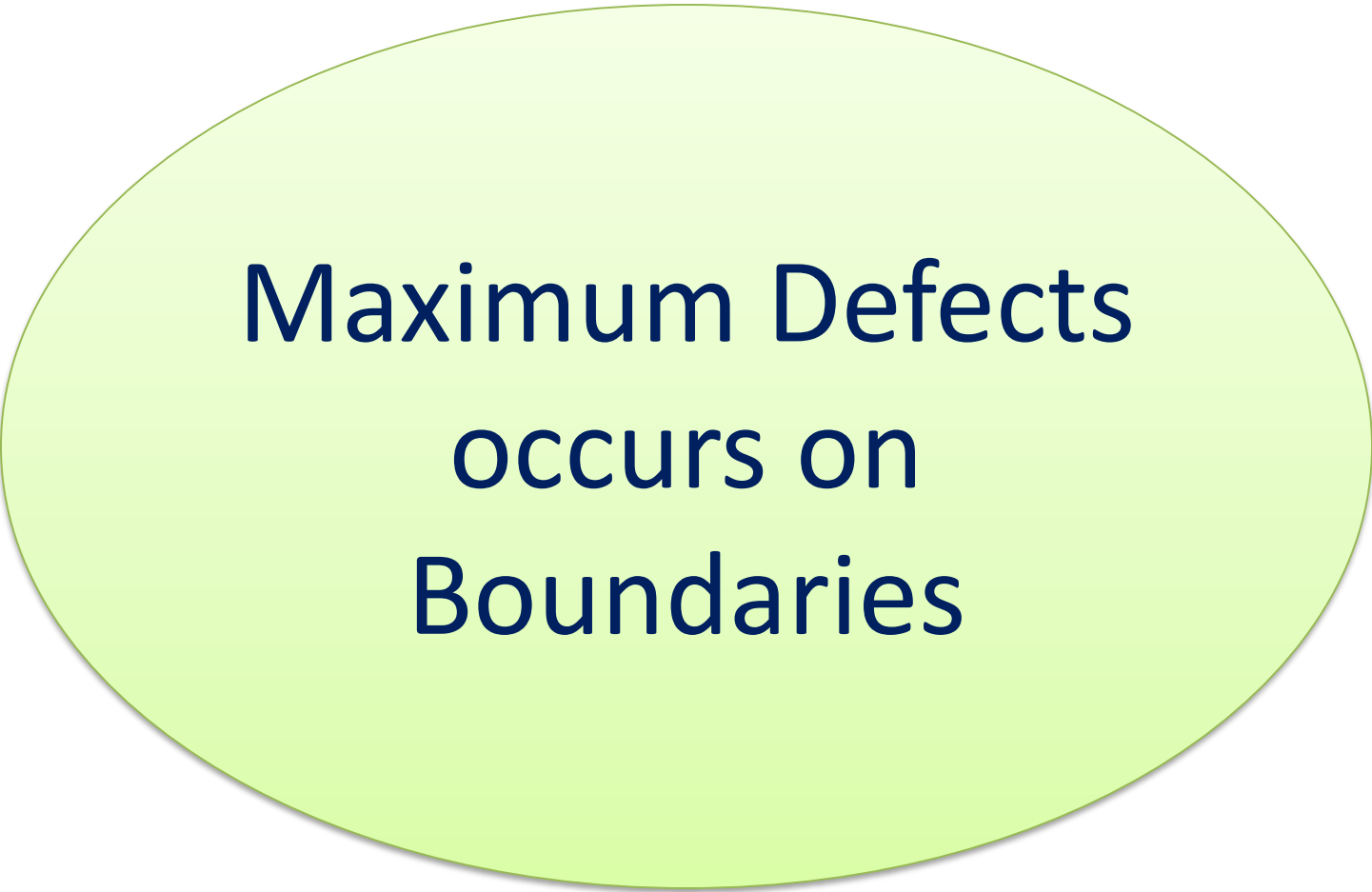
Taking each input condition and partitioning it into two or more groups:

- **Input Condition**
range of values x : 1-50
- **Valid equivalence class**
 $? < x < ?$
- **Invalid equivalence classes**
 $x < ?$
 $x > ?$



Equivalence Class -Limitations

- It is most suited to systems in which much of the input data takes on values within ranges or within sets.
- It makes the assumption that data in the same equivalence class is processed in the same way by the system



Maximum Defects
occurs on
Boundaries

Boundary Value Analysis

Boundary Value Analysis

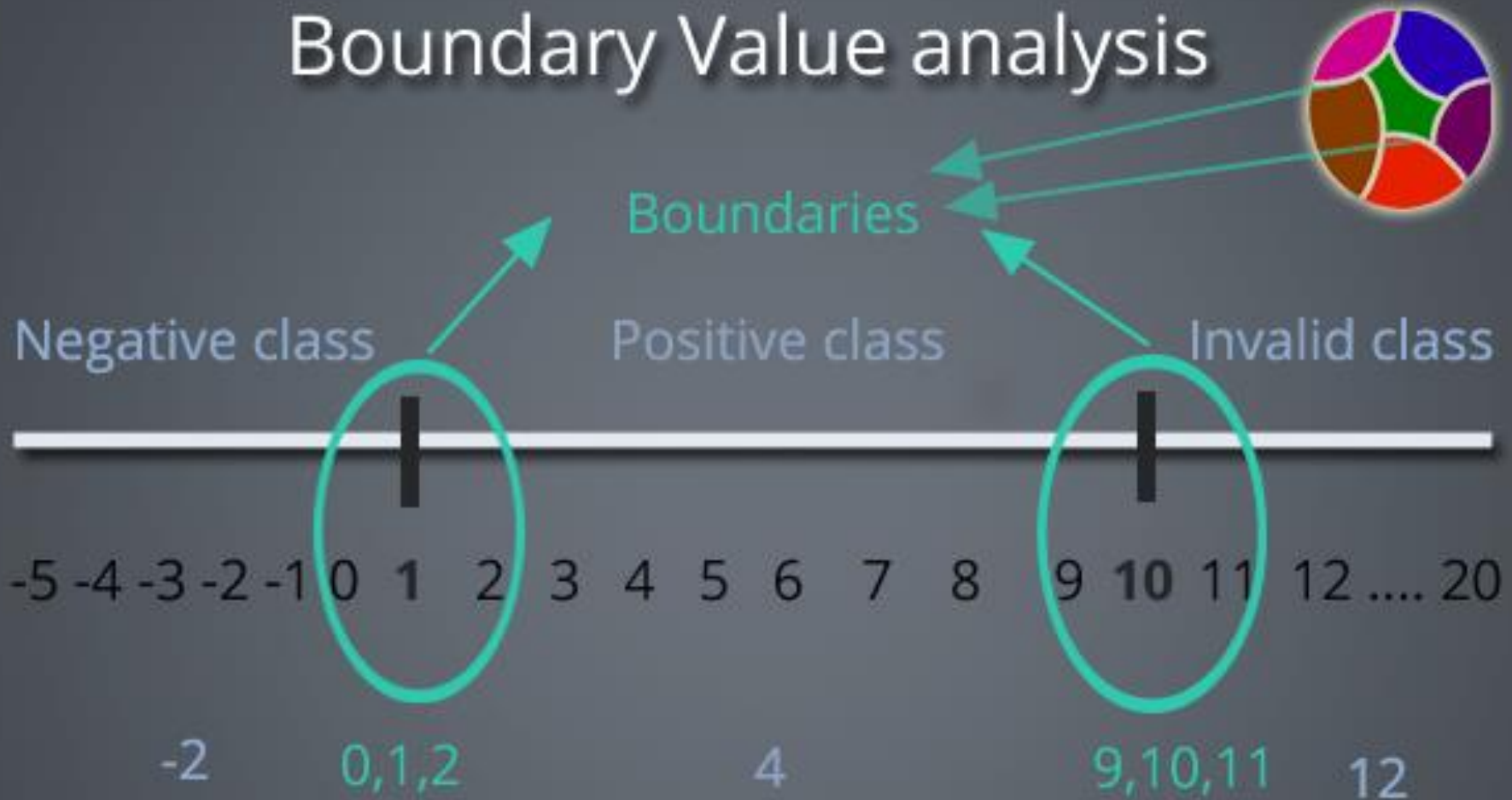
Boundary value testing focuses on the boundaries simply because that is where so many defects hide. The defects can be in the requirements or in the code.

The most **efficient** way of finding such defects, either in the requirements or the code, is through **inspection**

Boundary Value Analysis

1. Identify the **Ecs**
2. Identify the **boundaries** of each **EC**
3. Create **test cases** for each **boundary value** by choosing **one point** on the boundary, **one point** just **below** the boundary, and **one point** just above the boundary

Boundary Value analysis



Boundary Value Analysis -Example

Specification: the program accepts three to six inputs which are 5 digit integers greater than 10000.

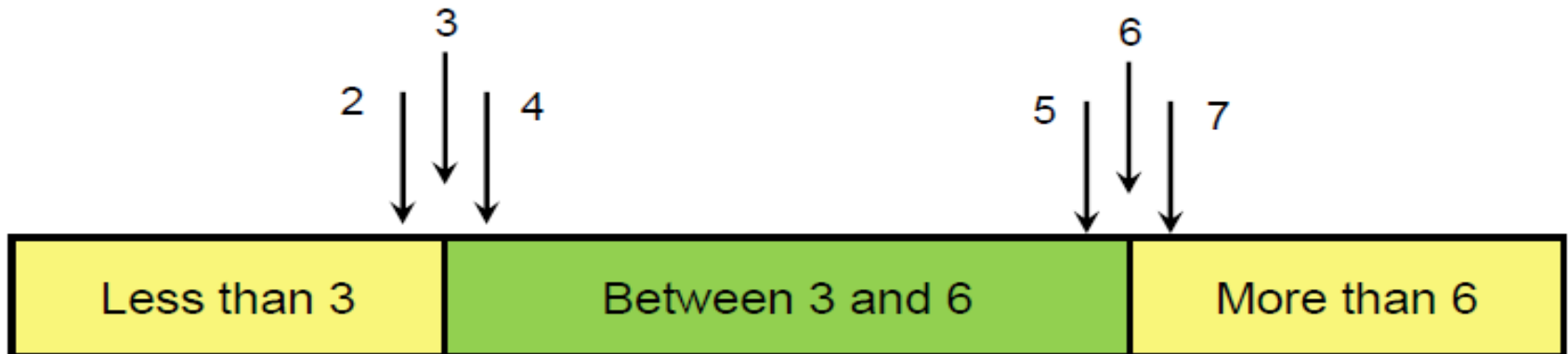
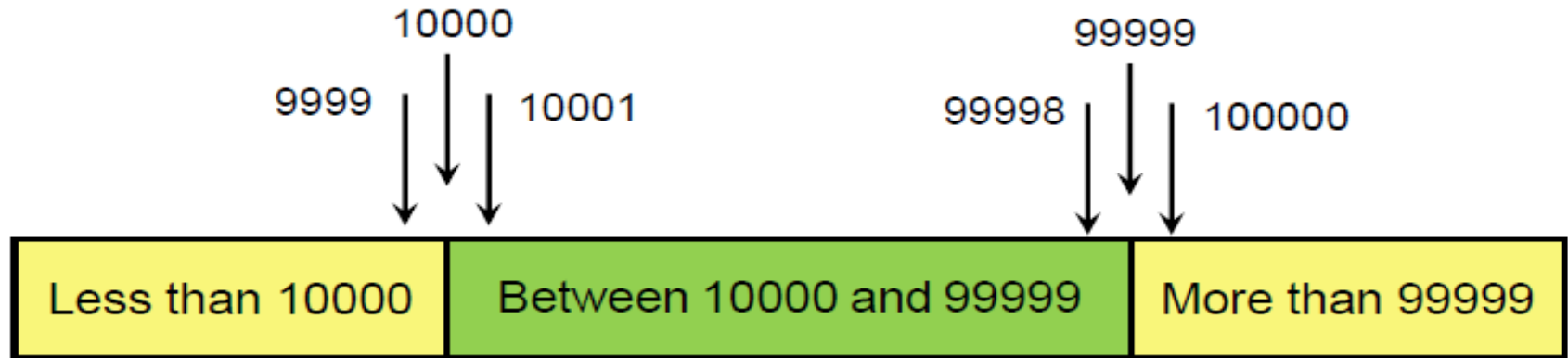
Input Values

Less than 10000	Between 10000 and 99999	More than 10000
-----------------	-------------------------	-----------------

Number of input values

Less than 3	Between 3 and 6	More than 6
-------------	-----------------	-------------

Boundary Value Analysis -Example



Boundary Value

- Boundary value testing is equally applicable at the **unit**, **integration**, **system**, and **acceptance** test levels
- All it requires are inputs that can be partitioned and boundaries that can be identified based on the system's requirements

Don't let your **programmers** check their own work; they'll miss their own errors.



Decision Table Testing

- An excellent tool for capturing certain kinds of system requirements and documenting internal system design
- Record complex business rules that a system must implement
- Also can serve as a guide to creating test cases

Decision Table Testing

- Used to layout in **tabular** form **all possible situations** which a decision may encounter and to specify which action to take in each of these situations
- A **matrix representation** of the **logic** of a decision
- Specifies the **possible conditions** and the **resulting actions**

Decision Table Testing

- **Condition**

- Condition describe the **conditions** or **factors** that will affect the decision or policy
- They are listed in the **upper section** of the decision table

- **Action**

- Action describe, in the form of statements, the possible policy **actions** or **decisions**
- They are listed in the **lower section** of the decision table

Decision Table Testing

■ Rules

- Rules describe which actions are to be taken under a specific combination of conditions
- They are specified by first inserting different combinations of condition attribute values and then putting X's in the appropriate columns of the **action section** of the table

Decision Table Format

	Rule 1	Rule 2	...	Rule P
Condition-1				
Condition-2				
...				
Condition-M				
Action-1				
Action-2				
...				
Action-N				

Decision Table Format

Inputs

	Rule 1	Rule 2	...	Rule P
Condition-1				
Condition-2				
...				
Condition-M				
Action-1				
Action-2				
...				
Action-N				

Expected Results

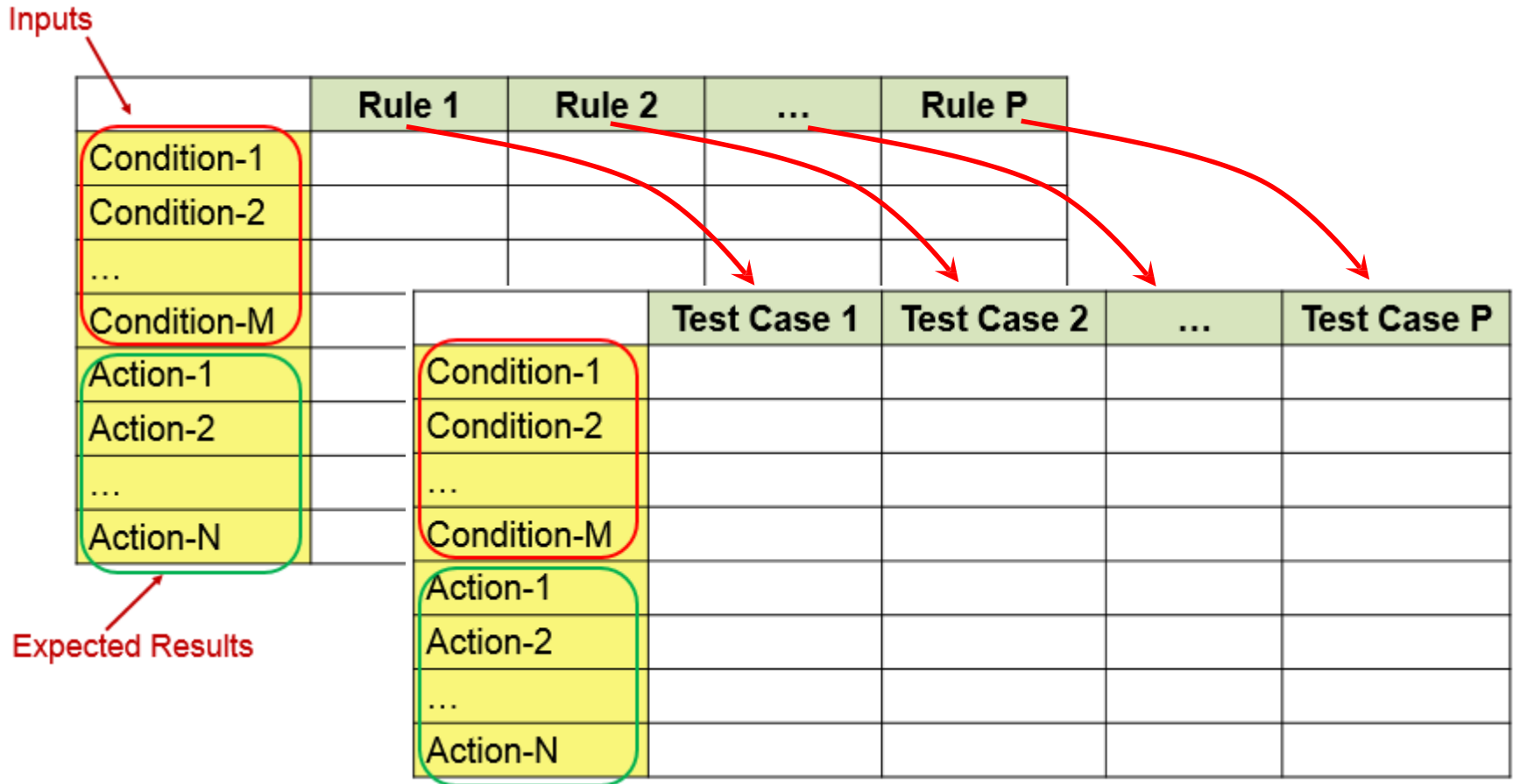
Decision Table - Don't Care Entry

	Rule 1	Rule 2	Rule 3	Rule 4
Condition-1	T	F	F	F
Condition-2	T	T	F	T
Condition-3	T	F	T	T
Action-1		—	X	—
Action-2	X		X	
Action-3		X		X

- "don't care" / "—" entry: the condition is irrelevant, or does not apply
- Limited entry decision tables: all the conditions are binary
- Extended entry decision tables: conditions are allowed to have several values
- Decision tables are deliberately declarative; no particular order is implied by the conditions, and selected actions do not occur in any particular order

Decision Table vs Test Case Table

A **decision table** converted to a **test case table**:



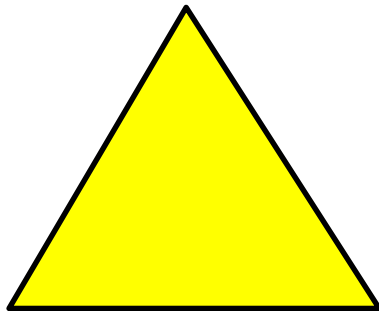
Decision Table – Triangle Program

- A program with three input integers, a, b, and c
- If a triangle then integers a, b, and c must satisfy:

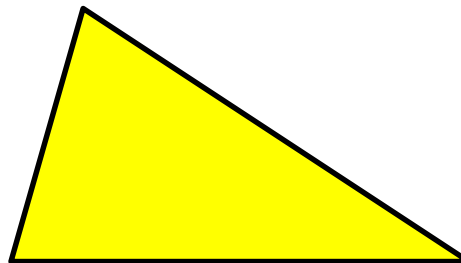
$$a < b + c$$

$$b < a + c$$

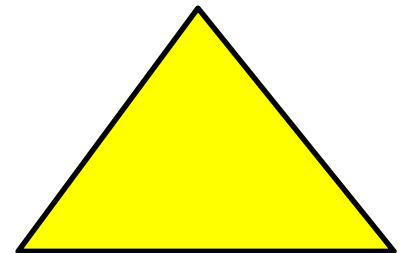
$$c < a + b$$



Isosceles



Scalene ($a \neq b \neq c$)



Equilateral ($a = b = c$)

Decision Table – Triangle Program

[illegible]

Decision Table – Triangle Program

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?									
C2: a = b?									
C3: a = c?									
C4: b = c?									
A1: Not a triangle									
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									

If not a triangle, pointless of looking at C2,C3, or C4 (rule 1)

Decision Table – Triangle Program

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	N								
C2: a = b?	—								
C3: a = c?	—								
C4: b = c?	—								
A1: Not a triangle	X								
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									

If not a triangle, pointless of looking at C2,C3, or C4 (rule 1)

Decision Table – Triangle Program

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	N								
C2: a = b?	—								
C3: a = c?	—								
C4: b = c?	—								
A1: Not a triangle	X								
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									

If not a triangle, pointless of looking at C2,C3, or C4 (rule 1)

If $a=c$ and $a=b$ then the negative entry in C4 is impossible (rule3)

Decision Table – Triangle Program

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	N	Y	Y						
C2: a = b?	–	Y	Y						
C3: a = c?	–	Y	Y						
C4: b = c?	–	Y	N						
A1: Not a triangle	X								
A2: Scalene									
A3: Isosceles									
A4: Equilateral		X							
A5: Impossible			X						

If not a triangle, pointless of looking at C2,C3, or C4 (rule 1)

If $a=c$ and $a=b$ then the negative entry in C4 is impossible (rule3)

Decision Table – Triangle Program

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	N	Y	Y	Y	Y	Y	Y	Y	Y
C2: a = b?	—	Y	Y	Y	Y	N	N	N	N
C3: a = c?	—	Y	Y	N	N	Y	Y	N	N
C4: b = c?	—	Y	N	Y	N	Y	N	Y	N
A1: Not a triangle	X								
A2: Scalene									X
A3: Isosceles					X		X	X	
A4: Equilateral		X							
A5: Impossible			X	X		X			

If not a triangle, pointless of looking at C2,C3, or C4 (rule 1)

If $a=c$ and $a=b$ then the negative entry in C4 is impossible (rule3)

Decision Table vs Test Case Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?									
C2: a = b?									
C3: a = c?									
C4: b = c?									
A1: Not a triangle									
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									
Test Case ID	a	b	c	Expected Output					
DT1									
DT2									
DT3									
DT4									
DT5									
DT6									
DT7									
DT8									
DT9									

$$a < b + c$$

$$b < a + c$$

$$c < a + b$$

Decision Table vs Test Case Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	F								
C2: a = b?	—								
C3: a = c?	—								
C4: b = c?	—								
A1: Not a triangle	X								
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									
Test Case ID	a	b	c	Expected Output					
DT1	4	1	2	Not a Triangle					
DT2									
DT3									
DT4									
DT5									
DT6									
DT7									
DT8									
DT9									

$$a < b + c$$

$$b < a + c$$

$$c < a + b$$

Decision Table vs Test Case Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	F	T	T						
C2: a = b?	—	F	T						
C3: a = c?	—	—	F						
C4: b = c?	—	—	—						
A1: Not a triangle	X	X	X						
A2: Scalene									
A3: Isosceles									
A4: Equilateral									
A5: Impossible									
Test Case ID	a	b	c	Expected Output					
DT1	4	1	2	Not a Triangle					
DT2	1	4	2	Not a Triangle					
DT3	1	2	4	Not a Triangle					
DT4									
DT5									
DT6									
DT7									
DT8									
DT9									

$$a < b + c$$

$$b < a + c$$

$$c < a + b$$

Decision Table vs Test Case Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8	Rule 9
C1: a, b, c form a triangle?	F	T	T	T					
C2: a = b?	—	F	T	T					
C3: a = c?	—	—	F	T					
C4: b = c?	—	—	—	T					
A1: Not a triangle	X	X	X						
A2: Scalene									
A3: Isosceles									
A4: Equilateral				X					
A5: Impossible									
Test Case ID	a	b	c	Expected Output					
DT1	4	1	2	Not a Triangle					
DT2	1	4	2	Not a Triangle					
DT3	1	2	4	Not a Triangle					
DT4	5	5	5	Equilateral					
DT5									
DT6									
DT7									
DT8									
DT9									

$$a < b + c$$

$$b < a + c$$

$$c < a + b$$

Nondeterministic Table

- Problem: when the action entries are different: e.g., when **C1** is true and both **C2** and **C3** are false, both rules 4 and 9 are applicable. We can make two observations:
 - Rules 4 and 9 are inconsistent
 - The decision table is nondeterministic
- Rules 4 and 9 are inconsistent because the action sets are different. The whole table is nondeterministic because there is no way to decide whether to apply rule 4 or rule 9
- Thus care should be taken when don't care entries used in a decision table

Nondeterministic Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6
C1	T	F	F	F	F	T
C2	–	T	T	F	F	F
C3	–	T	F	T	F	F
A1	X	X	X	–	–	–
A2	–	X	X	X	–	X
A3	X	–	X	X	X	–

Nondeterministic Table

	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6
C1	T	F	F	F	F	T
C2	–	T	T	F	F	F
C3	–	T	F	T	F	F
A1	X	X	X	–	–	–
A2	–	X	X	X	–	X
A3	X	–	X	X	X	–

Applicability

- Decision table testing can be used whenever the system must implement **complex business rules** when these rules can be represented as a **combination of conditions** and when these conditions have **discrete actions** associated with them

Task

- **Scenario:** A marketing company wishes to construct a decision table to decide how to treat clients according to three characteristics:
 - Gender
 - City Dweller
 - age group: A(under 30), B(between 30 and 60), C(over 60)
- The company has four products (W, X, Y and Z) to test market
 - Product **W** will appeal to **male city dwellers**.
 - Product **X** will appeal to **young males**.
 - Product **Y** will appeal to **Female middle aged** shoppers who **do not live in cities**.
 - Product **Z** will appeal to **all but older males**

1. Identify Conditions & Values

- The **three** data attributes tested by the conditions in this problem are
 - **Gender**, with values **M** and **F**
 - **City dweller**, with value **Y** and **N**
 - **Age group**, with values **A**, **B**, and **C**

2. Compute Maximum Number of Rules

- The maximum number of Rules is:

$$2 \times 2 \times 3 = 12$$

3. Identify Possible Actions

- The four actions are:
 - Market product **W**
 - Market product **X**
 - Market product **Y**
 - Market product **Z**

3. Identify Possible Actions

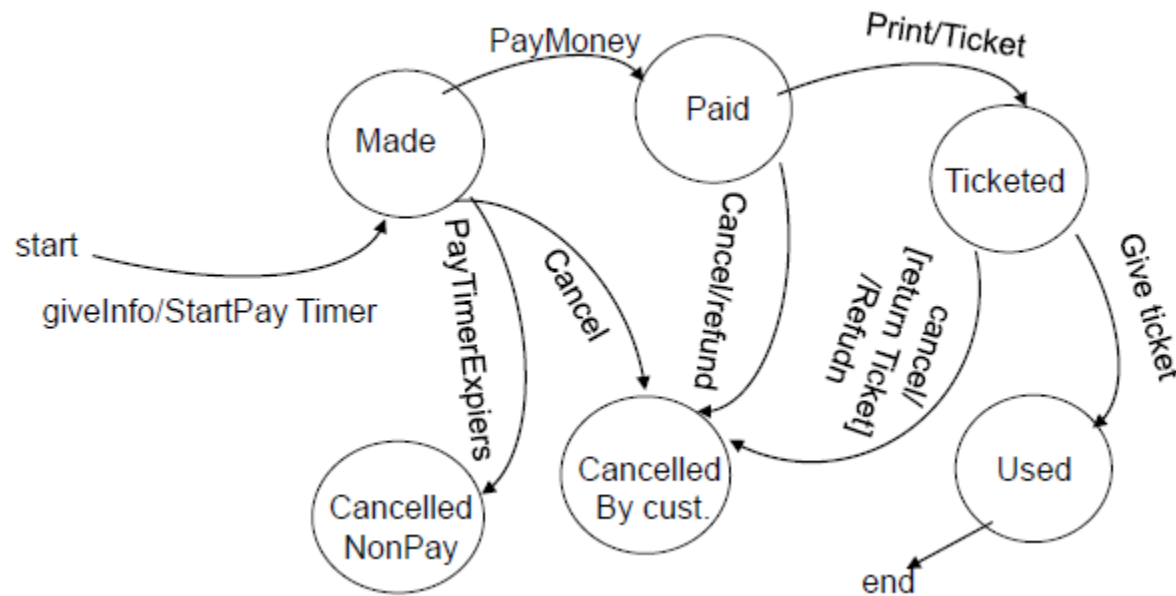
	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12
Gender	M	F	M	F	M	F	M	F	M	F	M	F
City	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N
Age	A	A	A	A	B	B	B	B	C	C	C	C

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12
W	X				X				X			
X	X		X									
Y								X				
Z	X	X	X	X	X	X	X	X		X		X

Advantage of decision table technique:

- 1.Any complex business flow can be easily converted into the test scenarios & test cases using this technique.
- 2.Such type of table are work iteratively, means the table created at the first iteration is used as input table for next tables. Such iteration can be carried out only if the initial table is unsatisfactory.
- 3.Simple to understand and everyone can use this method design the test scenarios & test cases.
- 4.It provide complete coverage of test cases which help to reduce the rework on writing test scenarios & test cases.
- 5.These tables guarantee that we consider every possible combination of condition values. This is known as its "completeness property".

State-Transition Testing



Current State	Event	Action	Next State
NULL	giveInfo	StartPayTimer	Made
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Applicability and Limitations

- Excellent to capture certain system requirements
- Not applicable when the system has no state or does not need to respond to real-time events from outside of the system

Stress Testing

Stress testing is a software testing activity that determines the robustness of software by testing beyond the limits of normal operation. Stress testing is particularly important for "mission critical" software, but is used for all types of software. Stress tests commonly put a greater emphasis on robustness, availability, and error handling under a heavy load, than on what would be considered correct behavior under normal circumstances.

Fuzzy Testing

Fuzz testing is a software testing technique using which a **random data** is given as the inputs to the system.

If the application fails, then those issues/defects are to be addressed by the system. In short, unexpected or random inputs might lead to unexpected results.

Progressive Testing?

Progressive testing also known as **incremental testing** is used to test modules one after the other. When an application with a hierarchy such as parent-child module is being tested, the related modules would need to be tested first.

This progressive approach testing method has three approaches:

- Top-down Approach
- Bottom-up Approach
- Hybrid Approach

Re-testing

Re-testing is executing a previously failed test against new software to check if the problem is resolved. After a defect has been fixed, re-testing is performed to check the scenario under the same environmental conditions. During Re-testing, testers look for granular details at the changed area of functionality.